## Program one assignment answer

### 1) *Problem one – convolution function*

#### *a) Create a subroutine or function that will perform convolution.*

This code can be seen in appendix A. Some notes about implementing the function in C++. Although getting the bounds of the two for loops was not hard, it was still not getting the correct results. However, using an if statement to check the bounds of the array proved to be effective in fixing the issue.

### 2) *Problem two – convolution with arrays*

Handwritten work for the following convolutions can be found at the end of the document in appendix B. The next figures will be comparing the MATLAB convolution to the numerical method used using C++.

#### *a) F1 * F1*

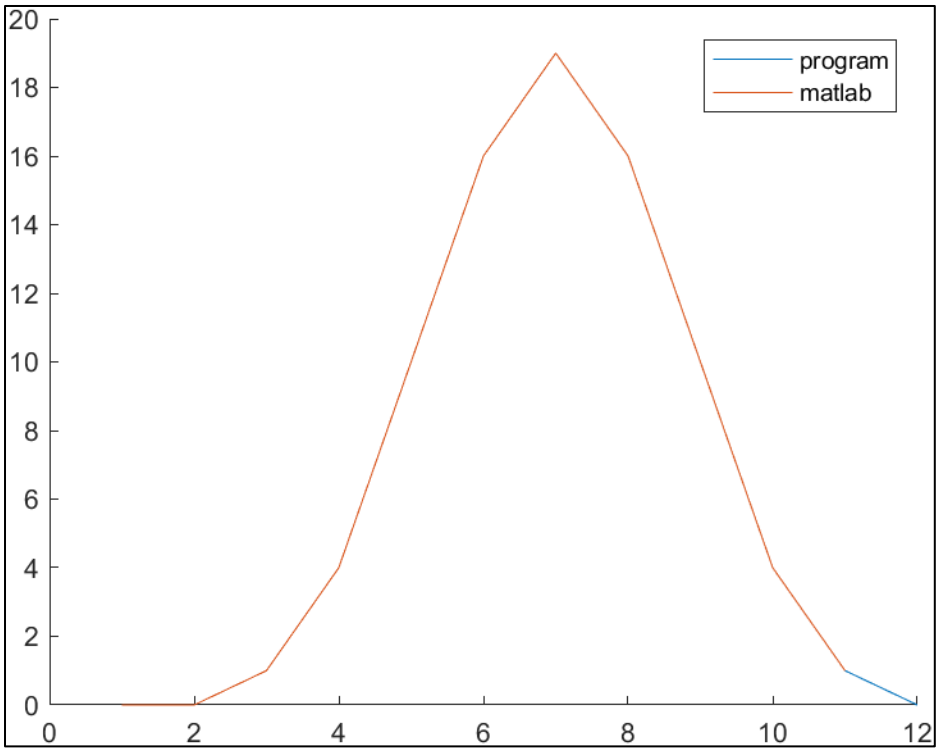



Figure 1: Convolution of F1 with F1 with MATLAB and programming plotting on the same plot.

***b) F1 * F2***

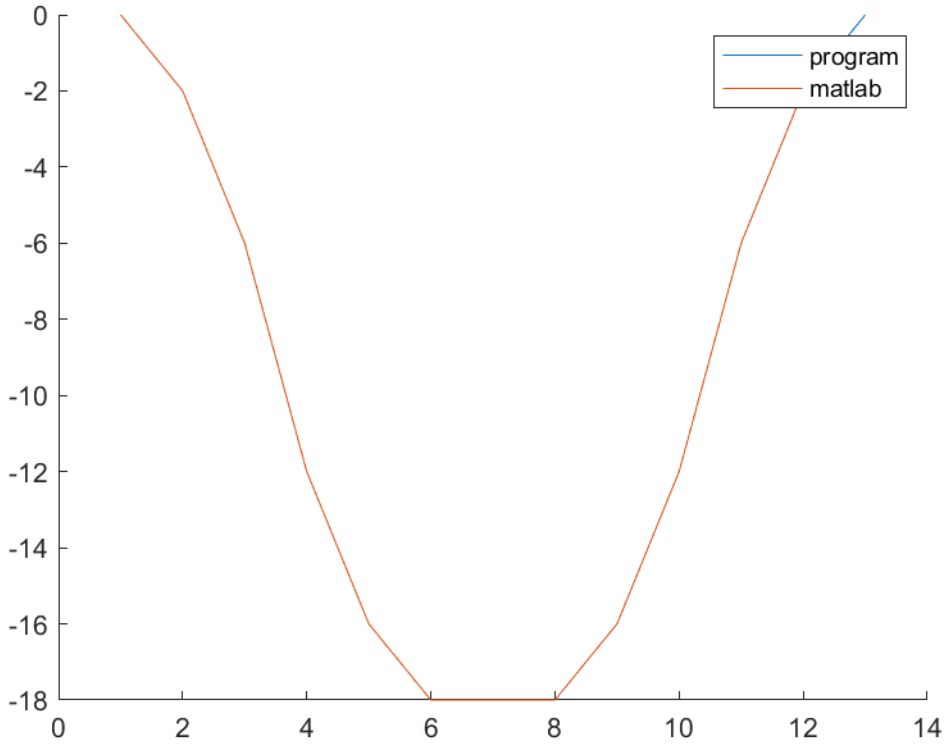



Figure 2: Convolution of F1 with F2 with MATLAB and programming plotting on the same plot.

***c) F1 * F3***

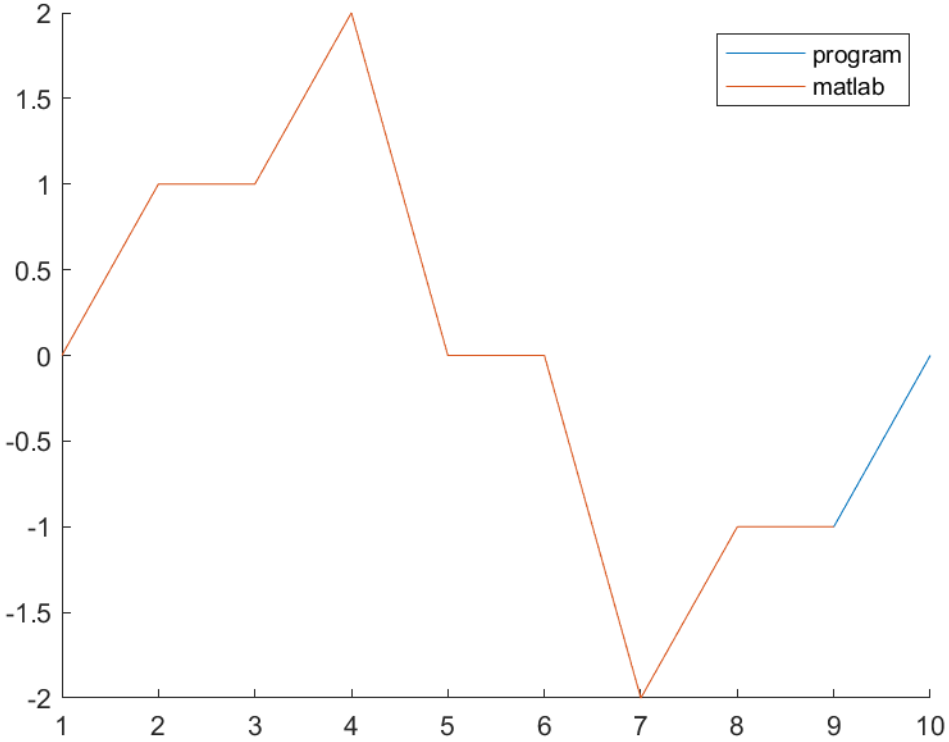



Figure 3: Convolution of F1 with F3 with MATLAB and programming plotting on the same plot.

***d) F2 * F3***

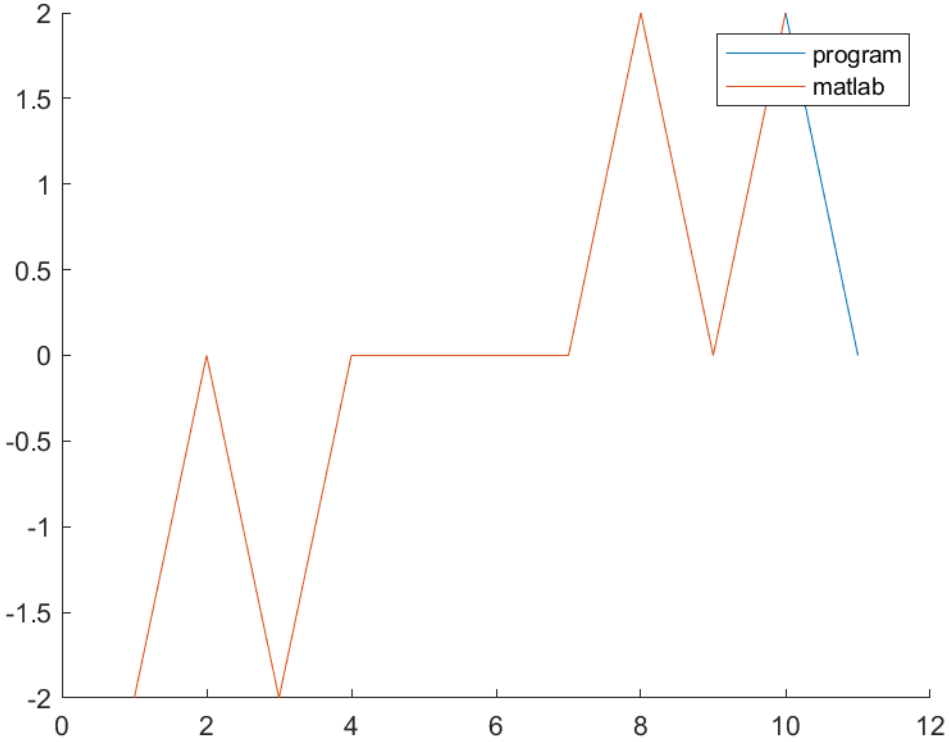



Figure 4: Convolution of F2 with F3 with MATLAB and programming plotting on the same plot.

***e) F1 * F4***

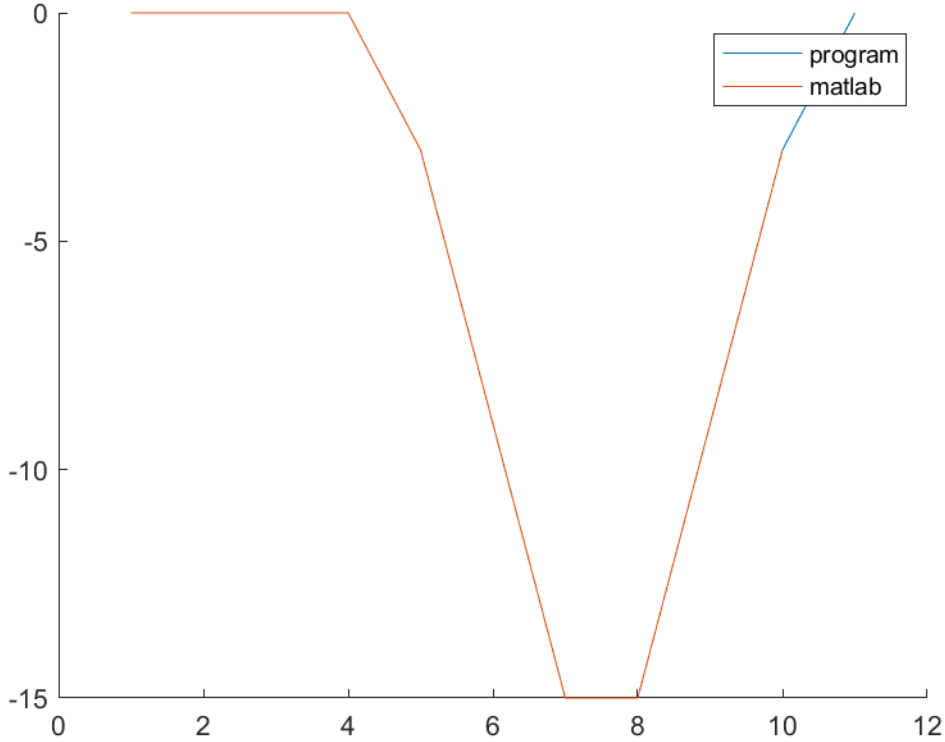



Figure 5: Convolution of F1 with F4 with MATLAB and programming plotting on the same plot.

### *3) Third order differential – solving for total solution*

$$(D^3 + 5D^2 + 12D + 15)y(t) = (D + 0.5)f(t)$$

$$y(0) = -3, y'(0) = 2, y''(0) = 1$$

$$f(t) = sin(3 * pi * t) * u(t)$$

When using numerical methods use $T = 0.001$ and have time be between $0 \leq t \leq 10$

### *a) Find the impulse response h(t) then compute and plot sampled values h[k] = h(kT)*

Handwritten work can be found at the end of the document in appendix B. The following figure shows the impulse response plotted using MATLAB.

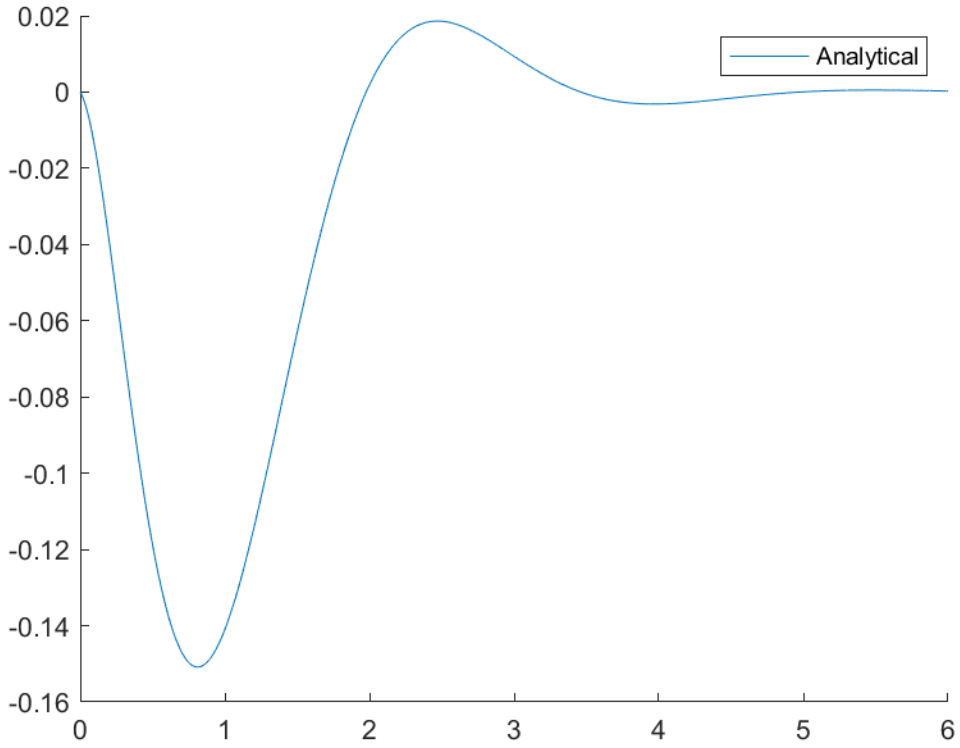



Figure 5: Analytical solution of h(t) plotted using MATLAB.

***b) Find the sampled values of the input function f[k] = f(kT) and plot them***

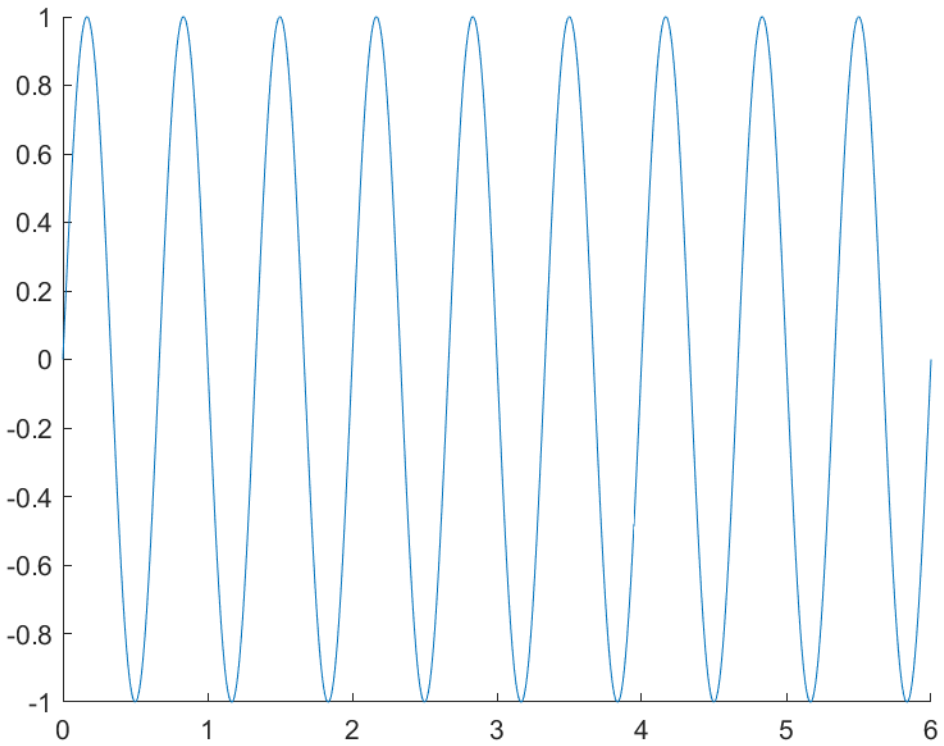



Figure 6: Plotting the discrete points of f(t) using MATLAB from t = 0 to t = 6

***c) The zero-state solution is the scaled convolution T(f[k] * h[k])***

***d) The zero-input solution is found using Program #1***

***e) The total solution is the sum of the zero-state solution and the zero-input solution.***

***f) Compute numerically and plot the total solution.***

To compute numerically the program made for the first assignment was used to get discrete values. Then using the array in the current program, then the program added the value from program 1 and added it to the array value in program 2.

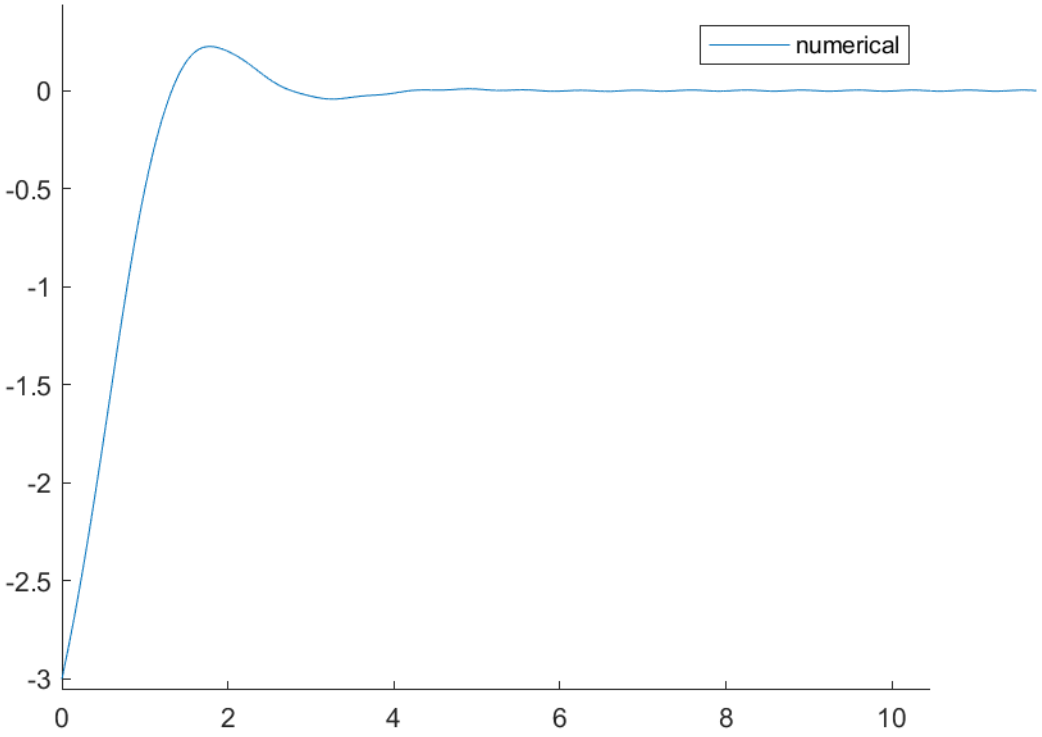



Figure 7: numerical total solution to the differential equation plotted in MATLAB.

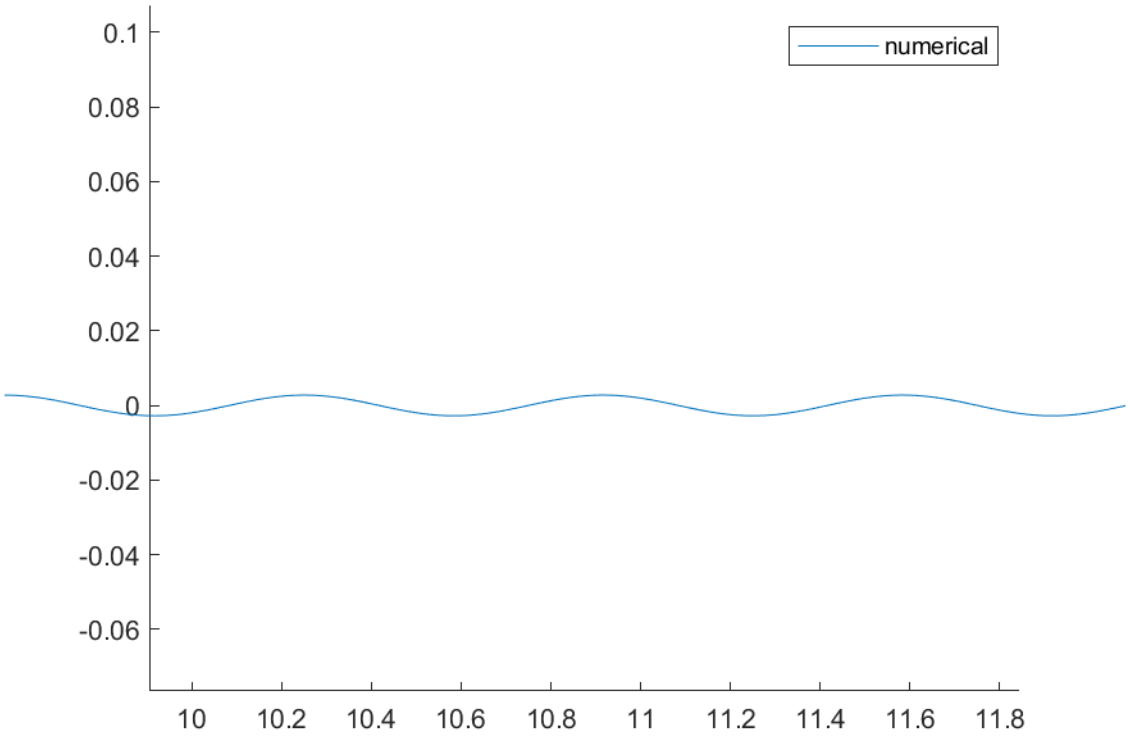



Figure 8: Close of the steady state behavior of the numerical solution plotted in MATLAB.

### *g) Find an analytical solution to the DE and plot it.*

Work solving the differential equation can be found in appendix B. The following is the plot of the total solution found.

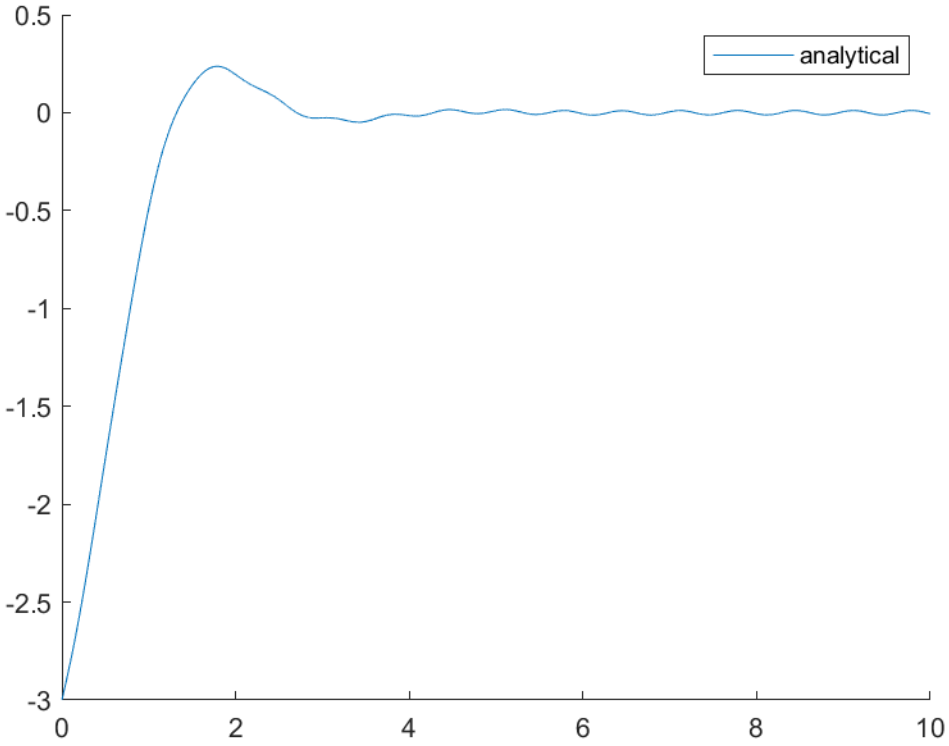



Figure 9: MATLAB plot of the total solution found analytically.

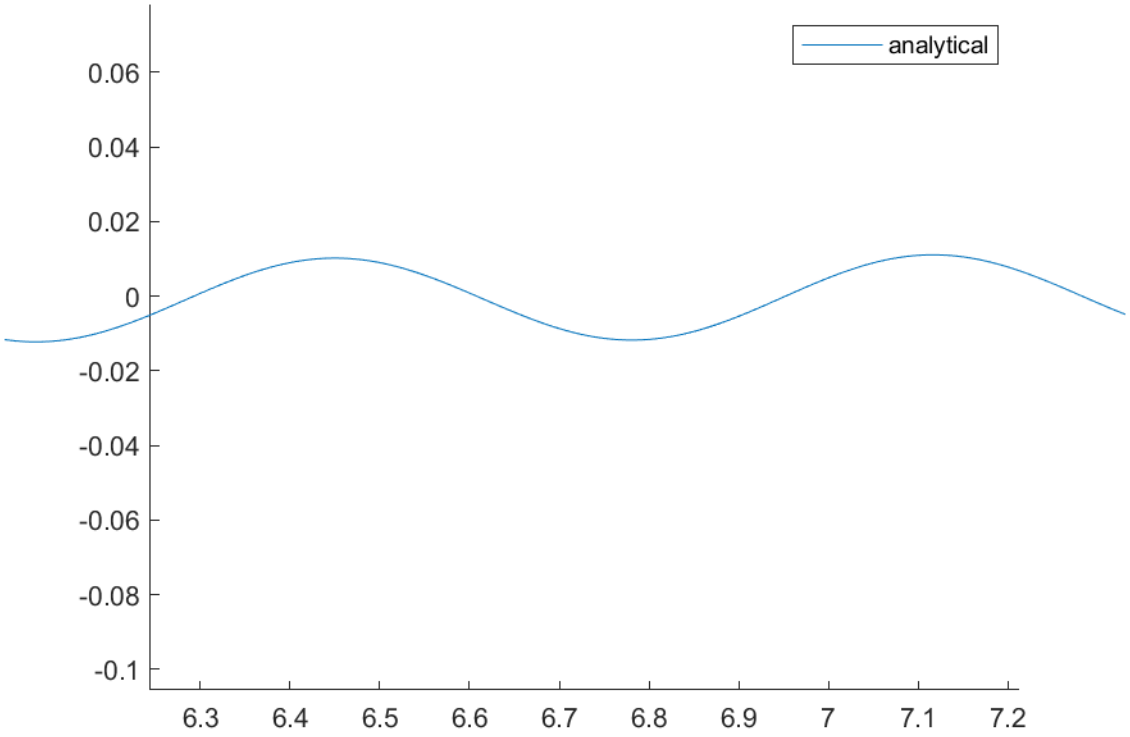



Figure 10: Close up of the total solution found to show steady state of differential equation.

## *h) Compare the analytical solution and the numerical solution.*

Although the results for analytical and numerical are close to each other. The steady state differs the most. The steady state for the numerical solution has less amplitude than the analytical solution. However, despite the phase shift both solutions have the same frequency for the end state behavior. Besides that, the numerical solution was able to approximate the solution.

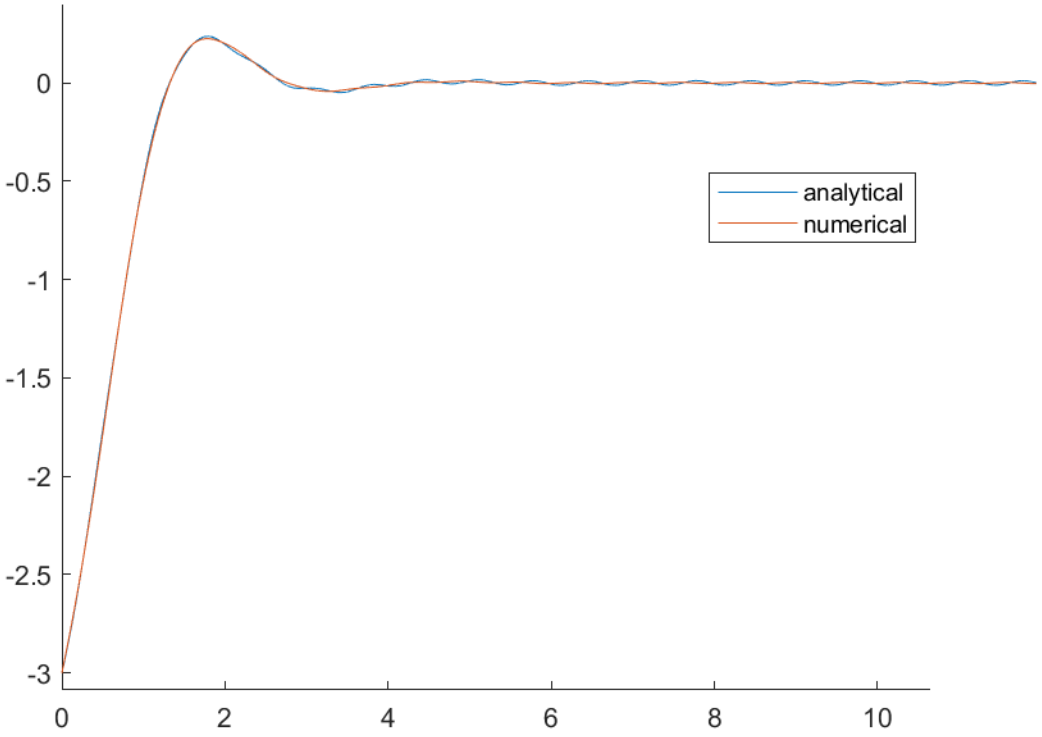



Figure 11: Numerical and analytical total solutions plotted using MATLAB.

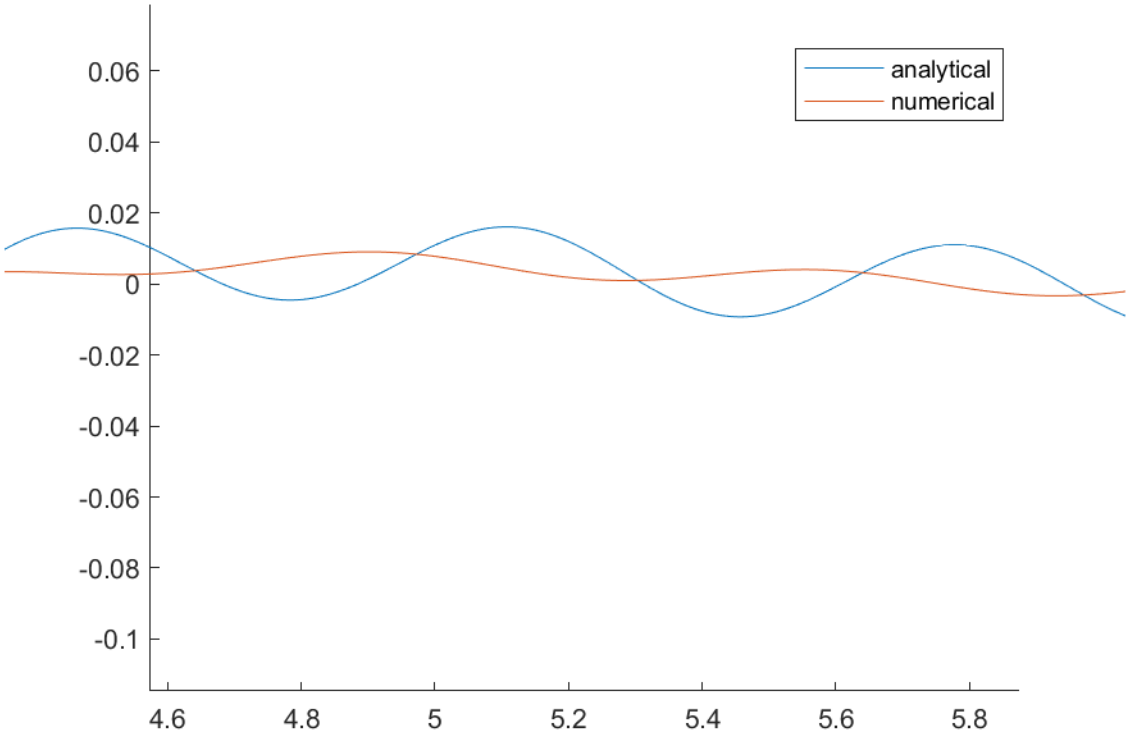



Figure 12: Close up of the two solutions to show the amplitude and phase shift difference.

### *4) Steady state*

The stead state amplitude for the analytical is .01 and the amplitude for the numerical solution is .004. This happens because the differential equation given represents a system that filters out this frequency of a sin wave. However, the cutoff frequency could be close to the frequency of the sin wave and that would explain why it is not completely filtered out.

# Appendix A – Code snippets

```
#include <iostream>

#include <fstream>

#include <vector>

using namespace std;


//implementation of convolution using array, makes it easier for larger arrays

void Convolution(const vector<double>& f1, const vector<double>& f2, vector<double>& result, int T) {

     //find length 1

    int len1 = f1.size();

     //find length 2

    int len2 = f2.size();

     //find length 3

    int kmax = len1 + len2 - 1;


     //resize vector

    result.resize(kmax);


    for (int k = 0; k < kmax; ++k) {

           //sets result to 0

        result[k] = 0;

           //perform the algebra for single time

        for (int m = 0; m < len1; ++m) {

            int index = k - m;

                //checks if it is within bounds

            if (index >= 0 && index < len2) {

                      //sum of multiplication

                result[k] += f1[m] * f2[index];

            }

        }
```

```
    }
}

//functions same as above but uses arrays rather than vector and vector
functions
int Conv(double* f1, int len1, double* f2, int len2, double*& y) {
    int kmax = len1 + len2;
    y = new double[kmax];

    for (int k = 0; k < kmax; k++) {
        y[k] = 0;
        for (int m = 0; m < len1; m++) {
            int index = k - m;
            if (index >= 0 && index < len2) {
                y[k] += f1[m] * f2[index];
            }
        }
    }
     return kmax;
}



int main() {
    // Read data from the first text file

     int t;
     int leny;
     double* result;

     //all the different arrays given in PDF
    double f2[] = {-2, -2, -2, -2, -2, -2, -2};
    int len2 = 7;
```

```
double f3[] = {1, -1, 1, -1};
int len3 = 4;

 double f1[] = {0,1,2,3,2,1};
 int len1 = 6;

 double f4[] = {0,0,0,-3,-3};
 int len4 = 5;

 //all output files for matlab
 ofstream outfile1("convolution_1");
 ofstream outfile2("convolution_2");
 ofstream outfile3("convolution_3");
 ofstream outfile4("convolution_4");
 ofstream outfile5("convolution_5");
 ofstream outfile6("convolution_6");


 //computes A
leny = Conv(f1, len1, f1, len1, result);
 for(t = 0; t < leny; t++){
       //outputs result and time to output file
       outfile1 << t + 1 << " " <<  result[t] << endl;
 }
 delete[] result;

 //computes B
 leny = Conv(f1, len1, f2, len2, result);
 for(t = 0; t < leny; t++){
       //outputs result and time to output file
       outfile2 << t + 1<< " " <<  result[t] << endl;
```

```
  }
  delete[] result;

  //computes C
  leny = Conv(f1, len1, f3, len3, result);
  for(t = 0; t < leny; t++){
        //outputs result and time to output file
        outfile3 << t + 1<< " " <<  result[t] << endl;
  }
  delete[] result;

  //computes D
  leny = Conv(f3, len3, f2, len2, result);
  for(t = 0; t < leny; t++){
        //outputs result and time to output file
        outfile4 << t + 1<< " " <<  result[t] << endl;
  }
  delete[] result;

  //computes E
  leny = Conv(f1, len1, f4, len4, result);
  for(t = 0; t < leny; t++){
        //outputs result and time to output file
        outfile5 << t + 1<< " " <<  result[t] << endl;
  }
delete[] result; // Don't forget to free the allocated memory

  //opens file that has transfer function values
ifstream file1("output.txt");
  //transfer function
vector<double> h;
  //dummy variable
```

```
double value;
  //dummy variable
  int T = 1;

  //while value is still reading from file
while (file1 >> value) {
       //gets value for transfer function
    h.push_back(value);
}

//opens file that has sin function values
ifstream file2("output2.txt");
  //sin function
vector<double> f;

  //while value is still reading from file
while (file2 >> value) {
       //gets value for transfer function
    f.push_back(value);
}

//creates convolution solution
vector<double> convolutionResult;
  //computes convolution
Convolution(h, f, convolutionResult, T);

// Write the result to an output text file
ofstream outputFile("convolution_result.txt");
  //sets current time and step value
double currentTime = 0.0;
double timeStep = .001;
```

```
      //f[k]*h[k] = T * sum(f[k] x h[k])
      for(int i = 0; i < convolutionResult.size(); i++){
            convolutionResult[i] *= .001;
      }

      //gets value from program 1 for zero-input solution
      ifstream file3("approximation2");
      vector<double> zero_input;

    for (const auto& resultValue : convolutionResult) {
            //gets value from program 1
            file3 >> value;
            //adds yZS and yZI
            double temp = resultValue + value;
            //outputs time and value
        outputFile << currentTime << " " << temp << endl;
            //increment time
        currentTime += timeStep;
    }

    return 0;
}
```

# Appendix B – Handwritten work